JS

JavaScript Phase 1 Foundations

Revision guide: the most important questions, code and concepts in one place.

What is inside

#	Part	Concepts covered
1	Variables and Data Types	var / let / const, template literals, declaration vs assignment, 7 primitives, typeof
2	Input, Conversion, Coercion	prompt(), Number(), implicit coercion, ++ and --, assignment operators
3	Operators and Logic	== vs ===, truthy / falsy, short-circuit && and ||, ternary
4	Strings, Numbers, Math	string methods, immutability, parseInt / parseFloat, Math, random integers
5	Control Flow	if / else if / else, nested if, switch and fall-through
6	Loops	for, while, do...while, for...of, break, continue
7	Mini Projects	FizzBuzz, guessing game, calculator, temperature converter, vowel counter
8	Cheat Sheet and Interview Q&A	comparison tables, common traps, quick answers

PART 1 | Variables and Data Types

Storing values, choosing the right keyword, and knowing what type you have.

1 Variables and Template Literals

Task: *Store a student's name, age, city and isStudent, then print one sentence using a template literal.*

```
const name = "Aman";
const age = 22;
const city = "Bhopal";
const isStudent = true;

console.log(
  `${name} is ${age} years old, lives in ${city}, ` +
  `and student status is ${isStudent}.`
);
// Aman is 22 years old, lives in Bhopal,
// and student status is true.
```

KEY Use **const** by default. Switch to **let** only when the value will change.

KEY Template literal = backticks + **${ }** to insert values.

DOES Runs any expression inside: `${a + b}`.

DOES NOT `${ }` does nothing inside normal quotes "" or ''.

2 Declaration, Initialization, Reassignment

Task: *Declare score without a value, assign 10, change to 25, then try to reassign a const.*

```
let score;               // declaration only
console.log(score);      // undefined

score = 10;              // initialization
score = 25;              // reassignment

const country = "India";
country = "USA";         // TypeError!

var x = 1;
var x = 2;               // redeclaration: ok for var
// let x = 3; (same scope) -> SyntaxError
```

KEY **Declaration** creates the variable. **Initialization** gives the first value. **Reassignment** changes it later.

DOES **let** can be reassigned. **const** must get a value at declaration.

DOES NOT **const** cannot be reassigned. **let** and **const** cannot be redeclared in the same scope.

REMEMBER An unassigned variable is **undefined**, not empty or null.

3 typeof and the 7 Primitive Types

Task: *Create one value of each primitive type and print its typeof.*

```
typeof "JavaScript"    // "string"
typeof 42              // "number"
typeof true            // "boolean"
typeof undefined       // "undefined"
typeof null            // "object"   <- historical bug
typeof 123n            // "bigint"
typeof Symbol("id")    // "symbol"

typeof NaN             // "number"
```

KEY 7 primitives: string, number, boolean, undefined, null, bigint, symbol. Everything else is an **object** (arrays, functions, dates).

REMEMBER **typeof null** is "object" by an old bug kept for compatibility. null is still a primitive.

REMEMBER **NaN** means Not-a-Number, yet its type is "number".

DOES NOT typeof cannot tell an array from an object.

PART 2 | Input, Conversion and Coercion

prompt() gives text. JavaScript converts types silently, so you must know the rules.

4 User Input and Explicit Conversion

Task: *Read two values with prompt() and print sum, difference, product, division, remainder and power.*

```
const a = Number(prompt("First number:"));
const b = Number(prompt("Second number:"));

console.log(a + b);     // 13   (for 10 and 3)
console.log(a - b);     // 7
console.log(a * b);     // 30
console.log(a / b);     // 3.3333333333333335
console.log(a % b);     // 1    (remainder)
console.log(a ** b);    // 1000 (power)
```

KEY **prompt() always returns a string.** Convert first with Number().

DOES NOT Without Number(), `"10" + "3"` gives "103" (joining text, not adding).

DOES **%** gives the remainder. ****** raises to a power.

REMEMBER Cancel on prompt() gives null; bad text gives NaN after Number().

5 Type Coercion Challenge

Task: *Predict the output of each line, then verify and explain the conversion.*

```
"5" + 3          // "53"  number -> string (joins)
"5" - 3          // 2     string -> number
"5" * "2"        // 10    both -> number
true + 1         // 2     true  -> 1
false + 1        // 1     false -> 0
null + 1         // 1     null  -> 0
undefined + 1    // NaN   undefined -> NaN
```

KEY **+** is special: if either side is a string, it joins text. **- * /** always do math.

KEY In math: true=1, false=0, null=0, undefined=NaN.

REMEMBER Explicit conversion = you do it (Number, String, Boolean). Implicit coercion = JavaScript does it for you.

6 Pre-Increment vs Post-Increment

Task: *Predict x, a and b, then write your own decrement example.*

```
let x = 5;
let a = x++;   // a gets 5, THEN x becomes 6
let b = ++x;   // x becomes 7, THEN b gets 7

console.log(x);  // 7
console.log(a);  // 5
console.log(b);  // 7

let n = 10;
let p = n--;   // p = 10, n = 9
let q = --n;   // n = 8,  q = 8
```

KEY **x++** (post): use the old value, then add 1.

KEY **++x** (pre): add 1 first, then use the new value.

REMEMBER Same logic for **--**. Alone on a line, x++ and ++x behave the same; the difference shows only when the result is used.

7 Assignment Operators

Task: *Start with balance = 1000 and apply +, -, *, / and % using only assignment operators.*

```
let balance = 1000;

balance += 500;    // 1500
balance -= 200;    // 1300
balance *= 2;      // 2600
balance /= 4;      // 650
balance %= 300;    // 50  (650 = 2*300 + 50)

console.log(balance);  // 50
```

KEY a `+=` b means a `=` a `+` b. Same for -=, *=, /=, %=, **=.

DOES Shorter code, and the variable is only written once.

DOES NOT Cannot be used on a **const** variable.

PART 3 | Operators and Logic

Comparing values, truthy and falsy, short-circuiting and the ternary operator.

8 == vs ===

Task: *Predict and explain each comparison.*

```
5 == "5"            // true   (coerced to same type)
5 === "5"           // false  (number vs string)
0 == false          // true
0 === false         // false
"" == false         // true
"" === false        // false
null == undefined   // true   (special rule)
null === undefined  // false
```

KEY **==** loose: converts types first, then compares. **===** strict: compares value AND type.

REMEMBER Use **===** and **!==** in normal code. Results are predictable.

DOES NOT == has surprising rules; do not rely on them.

9 Truthy, Falsy and Short-Circuiting

Task: *Build fallback values with ||, then predict the output of && and ||.*

```
let userName = "", enteredAge = 0, customTheme = null;

userName    = userName    || "Guest";   // "Guest"
enteredAge  = enteredAge  || 18;        // 18
customTheme = customTheme || "light";   // "light"

"hello" && "world"   // "world"  (all truthy: last)
0 && "hello"         // 0        (first falsy)
"" || "default"      // "default"(first truthy)
"user" || "guest"    // "user"   (first truthy)
```

KEY **6 falsy values:** false, 0, "", null, undefined, NaN. Everything else is truthy.

KEY **||** returns the first truthy value. **&&** returns the first falsy value, or the last value if all are truthy.

DOES They return real operand values, not always true or false.

REMEMBER `"0"`, `"false"`, `[]`, `{}` are all **truthy**.

DOES NOT || also replaces a valid 0 (an age of 0 becomes 18).

10 Ternary Operator

Task: *Take an age with prompt() and store Adult (18 or above) or Minor.*

```
const age = Number(prompt("Enter your age:"));

const status = age >= 18 ? "Adult" : "Minor";

console.log(status);

// Syntax:
// condition ? valueIfTrue : valueIfFalse
```

KEY Ternary is an **expression**: it produces a value, so you can store it directly.

DOES Best for simple two-way choices.

DOES NOT Avoid nesting many ternaries; use if / else if instead.

PART 4 | Strings, Numbers and Math

Cleaning text, searching inside it, converting to numbers and using Math.

11 String Cleanup and Search

Task: *Trim and change case, then search and extract from "JavaScript is powerful".*

```
let text = "   JavaScript God Mode   ";
text.length                 // 25
text.trim()                 // "JavaScript God Mode"
text.trim().length          // 19
text.trim().toUpperCase()   // "JAVASCRIPT GOD MODE"
text.trim().toLowerCase()   // "javascript god mode"

let s = "JavaScript is powerful";
s.includes("Script")        // true
s.indexOf("is")             // 11
s.slice(0, 10)              // "JavaScript"
s.substring(14, 22)         // "powerful"
s.replace("powerful", "awesome")
s.startsWith("Java")        // true
s.endsWith("powerful")      // true
s.charAt(0)  /  s[0]        // "J"
```

KEY **Strings are immutable.** Every method returns a NEW string; the original stays unchanged.

REMEMBER Save the result: `const t = text.trim()`. Calling trim() alone changes nothing.

DOES slice(start, end) and substring(start, end) both stop BEFORE end.

DOES NOT replace() changes only the first match. indexOf() gives -1 if nothing is found.

DOES NOT You cannot do `s[0] = "X"`; it is silently ignored.

12 Number Conversion and Validation

Task: *From "499px", "4.8stars" and "hello" get 499, 4.8 and NaN.*

```
parseInt("499px")        // 499
parseFloat("4.8stars")   // 4.8
Number("hello")          // NaN
Number("499px")          // NaN  (whole string must be a number)

Number.isInteger(499)    // true
Number.isInteger(4.8)    // false

typeof NaN               // "number"
NaN === NaN              // false
```

KEY **Number()** is strict: the whole string must be valid. **parseInt / parseFloat** read from the start and stop at the first bad character.

DOES parseInt gives whole numbers only; parseFloat keeps decimals.

DOES NOT parseInt("px499") is NaN: parsing must start with a digit.

REMEMBER Check NaN with **Number.isNaN(x)**, never with ===.

13 Math Methods and Random Integers

Task: *Round -4.7 in four ways, find max/min, and generate a dice roll and a random number between min and max.*

```
Math.abs(-4.7)     // 4.7
Math.round(-4.7)   // -5
Math.floor(-4.7)   // -5   (always DOWN)
Math.ceil(-4.7)    // -4   (always UP)
Math.max(12, 45, 7, 31)   // 45
Math.min(12, 45, 7, 31)   // 7
Math.sqrt(144)            // 12
Math.pow(2, 10)           // 1024

// Dice: 1 to 6
Math.floor(Math.random() * 6) + 1

// Any range, both ends included
let min = 20, max = 30;
Math.floor(Math.random() * (max - min + 1)) + min
```

KEY **Math.random()** gives a decimal from 0 up to (but NOT including) 1.

KEY Formula: `floor(random * (max - min + 1)) + min`. The **+ 1** makes max possible.

REMEMBER floor on negatives goes to the smaller number: floor(-4.7) = -5, not -4.

DOES NOT Math.round() alone gives uneven odds for the first and last numbers; use floor.

PART 5 | Control Flow

Making decisions with if / else if / else and switch.

14 Grade Calculator and Nested if

Task: *Print grade A to F, or Invalid marks. Then check driving eligibility with a nested if.*

```
const marks = Number(prompt("Enter marks:"));

if (marks < 0 || marks > 100) {
  console.log("Invalid marks");   // check bad input FIRST
} else if (marks >= 90) {
  console.log("A");
} else if (marks >= 80) {
  console.log("B");
} else if (marks >= 70) {
  console.log("C");
} else if (marks >= 60) {
  console.log("D");
} else {
  console.log("F");
}

// Nested if
if (age >= 18) {
  if (hasLicence) console.log("You can drive");
  else console.log("You need a licence");
} else {
  console.log("Not eligible to drive");
}
```

KEY The chain stops at the **first true** condition. Order from highest to lowest range so each check is simple.

DOES Validate input first, so impossible values never reach the grades.

KEY Use a **nested if** when the second question only matters after the first is true.

DOES NOT Only ONE branch of an if / else if chain ever runs.

15 switch and Fall-Through

Task: *Take a number 1 to 7 and print Weekday for 1-5 and Weekend for 6-7, using intentional fall-through.*

```
const day = Number(prompt("Day (1-7):"));

switch (day) {
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    console.log("Weekday");   // 1 to 5 share this code
    break;
  case 6:
  case 7:
    console.log("Weekend");   // 6 and 7 share this code
    break;
  default:
    console.log("Invalid day");
}
```

KEY switch compares with **strict equality (===)**: the string "6" will not match case 6.

KEY **Fall-through:** without break, execution continues into the next case. Grouping empty cases uses this on purpose.

REMEMBER Forgetting **break** by mistake is a classic bug. Add a name per day with its own case + break if you also need Monday, Tuesday...

DOES **default** handles every unmatched value.

DOES NOT switch is poor for ranges (marks >= 90); use if / else if there.

PART 6 | Loops

Repeating work: pick the loop that matches the problem.

16 for Loop: Multiplication Table

Task: *Print the table of any number from 1 to 10 using a for loop and template literals.*

```
const n = Number(prompt("Enter a number:"));

for (let i = 1; i <= 10; i++) {
  console.log(`${n} x ${i} = ${n * i}`);
}

// 5 x 1 = 5
// 5 x 2 = 10
// ...
// 5 x 10 = 50
```

KEY for (**start**; **condition**; **update**). Best when you know how many times to repeat.

DOES Checks the condition BEFORE every round, including the first.

DOES NOT Forgetting the update (i++) or a wrong condition creates an infinite loop.

17 while, do...while, for...of, break, continue

Task: *Four small tasks that show how each loop and jump statement behaves.*

```
// A. while: checks first
let i = 1;
while (i <= 10) { console.log(i); i++; }

// B. do...while: body runs once even if false
let x = 10;
do { console.log(x); x++; } while (x < 5);  // prints 10

// C. for...of: each character of a string
for (let ch of "JavaScript") console.log(ch);

// D. break + continue (1 to 20)
for (let n = 1; n <= 20; n++) {
  if (n === 17) break;        // stop the whole loop
  if (n % 3 === 0) continue;  // skip this round only
  console.log(n);
}
// 1 2 4 5 7 8 10 11 13 14 16
```

KEY **while**: condition first, may run 0 times. **do...while**: body first, runs at least once.

KEY **for...of** gives each VALUE of an iterable (a string, an array).

KEY **break** ends the loop. **continue** skips only the current round.

REMEMBER Update the counter before using continue in a while loop, or it can loop forever.

DOES NOT for...of does not give indexes. Use a normal for loop when you need i.

PART 7 | Mini Projects

Combine everything: input, conversion, conditions and loops.

18 FizzBuzz

Task: *Print 1 to 50: multiples of 3 give Fizz, of 5 give Buzz, of both give FizzBuzz.*

```
for (let i = 1; i <= 50; i++) {
  if (i % 3 === 0 && i % 5 === 0) {
    console.log("FizzBuzz");   // most specific FIRST
  } else if (i % 3 === 0) {
    console.log("Fizz");
  } else if (i % 5 === 0) {
    console.log("Buzz");
  } else {
    console.log(i);
  }
}
```

KEY **Order matters.** 15, 30 and 45 divide by 3 AND 5; if the Fizz check came first, FizzBuzz would never print.

REMEMBER Rule: put the most specific condition first, the most general last.

19 Number Guessing Game

Task: *Pick a secret number from 1 to 100, keep asking, give Too high / Too low, and count attempts.*

```
const secret = Math.floor(Math.random() * 100) + 1;
let attempts = 0;
let guess;

do {
  guess = Number(prompt("Guess (1-100):"));
  attempts++;
  if (guess > secret) {
    console.log("Too high!");
  } else if (guess < secret) {
    console.log("Too low!");
  }
} while (guess !== secret);

console.log(`You got it in ${attempts} attempts!`);
```

KEY **do...while** fits because the user must guess at least once.

DOES Declare **guess** outside the loop so the while condition can see it.

DOES NOT A let declared inside the do { } block is not visible in the while ( ) check.

20 Simple Calculator

Task: *Take two numbers and an operator; block division by zero; reject unknown operators.*

```
const num1 = Number(prompt("First number:"));
const op   = prompt("Operator (+, -, *, /):");
const num2 = Number(prompt("Second number:"));

let result;
if (op === "+")      result = num1 + num2;
else if (op === "-") result = num1 - num2;
else if (op === "*") result = num1 * num2;
else if (op === "/") {
  result = num2 !== 0 ? num1 / num2
                      : "Cannot divide by zero";
} else {
  result = "Invalid operator";
}
console.log("Result:", result);
```

KEY Convert numbers with **Number()**, but keep the operator as a string.

DOES Handles + - * /, division by zero, and bad operators.

DOES NOT JavaScript does not throw on 5 / 0 (it gives Infinity), so you must check it yourself.

21 Temperature Converter and Vowel Counter

Task: *Convert C to F or F to C (2 decimals, accept c / f). Count vowels with toLowerCase, for...of and includes.*

```
const t = Number(prompt("Temperature:"));
const unit = prompt("C or F?").toUpperCase();

if (unit === "C") {
  console.log(((t * 9) / 5 + 32).toFixed(2) + " F");
} else if (unit === "F") {
  console.log((((t - 32) * 5) / 9).toFixed(2) + " C");
} else {
  console.log("Invalid unit");
}

// Count vowels
const str = prompt("Enter text:").toLowerCase();
let count = 0;
for (let ch of str) {
  if ("aeiou".includes(ch)) count++;
}
console.log(count);   // "JavaScript" -> 3
```

KEY **toUpperCase()** on the unit makes c and C behave the same.

REMEMBER **toFixed(2)** returns a **string**, not a number.

KEY Lowercase the input first, so you only check "aeiou" once.

DOES includes() on a string checks if one character or word is present.

PART 8 | Cheat Sheet and Interview Q&A;

Last-minute revision: comparison tables, traps and short answers.

var vs let vs const

Keyword	Reassign	Redeclare (same scope)	Scope	Use it when
var	Yes	Yes	Function	Avoid in modern code
let	Yes	No	Block	The value will change
const	No	No	Block	Default choice

Which loop should I use?

Loop	Checks condition	Minimum runs	Best for
for	Before each round	0	Known number of repeats (counters, tables)
while	Before each round	0	Repeat until something happens
do...while	After each round	1	Must run once (guessing game, menus)
for...of	Goes through values	0	Characters of a string, items of an array

Common traps (fast revision)

Trap	What really happens
prompt() input	Always a string. Convert with Number() before math.
"5" + 3 vs "5" - 3	"53" (join) vs 2 (math). Only + joins text.
typeof null / typeof NaN	"object" (old bug) / "number".
null vs undefined	undefined = no value given. null = intentionally empty.
Math.floor(-4.7)	-5. It moves toward the smaller number.
Forgot break in switch	Execution falls through into the next case.
FizzBuzz order	Check the "both" case before 3 or 5 alone.
toFixed()	Returns a string, not a number.
NaN === NaN	false. Use Number.isNaN().

Interview quick answers

Question	Short answer
What is JavaScript and its engine?	A language for web interactivity that also runs on servers (Node.js), mobile and desktop. The engine runs the code: V8 (Chrome, Node.js), SpiderMonkey (Firefox), JavaScriptCore (Safari).
Primitive types?	string, number, boolean, null, undefined, symbol, bigint. The main non-primitive type is object.
Falsy values?	false, 0, "", null, undefined, NaN.
Are strings mutable?	No. Methods like trim() and toUpperCase() return a new string.
Do && and || return booleans?	Not always. They return one of the operands (short-circuit evaluation).
Number vs parseInt vs parseFloat?	Number needs the whole string valid. parseInt / parseFloat read the leading number and ignore the rest.
if / else if vs switch?	if for ranges and mixed conditions; switch for one value against many exact matches.
break vs continue?	break ends the whole loop. continue skips to the next round.